2、崩溃治理 —— 开发中解决过最难的问题是什么?

其实从整个项目的崩溃治理展开来讲，我认为就是对APM（应用性能管理）平台的建设。最重要的还是从原理上入手，然后分层次、分优先级进行处理：

首先了解我们项目常见的崩溃有哪些。

然后如何对这些崩溃进行捕获和定位。

最终才是预防和治理。

崩溃的分类

按捕获层级上来划分的话：

- 可捕获的分类（90%～95%）：
  - **Mach Exception**：最底层的硬件或内核崩溃；
    - EXC_BAD_ACCESS, EXC_BAD_INSTRUCTION, EXC_ARITHMETIC, EXC_BREAKPOINT
  - **BSD Signal**崩溃：中间层，有Mach Exception转换而来；
    - SIGSEGV, SIGBUS, SIGABRT, SIGILL, SIGFPE, SIGSYS, SIGTRAP
  - **NSException**崩溃：应用层，OC异常；
    - NSRangeException, NSInvalidArgumentException等
    - 未捕获时 → SIGABRT → Signal层

> 注意：
>
> - 1、这些崩溃并不是完全层级分明的，会相互转化！大多数Mach Exception会转化成BSD Signal崩溃。
>
> 代码块
>
> ```
> // Mach异常 → BSD信号 转换关系
> // 内核中的转换映射
> EXC_BAD_ACCESS      → SIGSEGV (SIGBUS)
> EXC_BAD_INSTRUCTION → SIGILL
> EXC_ARITHMETIC      → SIGFPE
> EXC_BREAKPOINT      → SIGTRAP
> EXC_SOFTWARE        → SIGABRT
> ```

> - 2、swift的崩溃不属于NSException，它是一个独立的错误机制。而是主要在Signal层被捕获。

- 无法捕获的分类（5%～10%）：
  - **Watchdog**超时（系统强杀）
    - 启动/退出超时
  - **OOM / Jetsem** （内存强杀）
    - 内存耗尽被杀（需要FOOM检测）

崩溃的捕获

KSCrash等库如何捕获异常崩溃?

其实iOS崩溃体系整体是有3层对应的异常通知

层级	名称	来源	典型错误	捕获方式
① Mach 层（内核级）	Mach Exception	Darwin Kernel	EXC_BAD_ACCESS、EXC_CRASH、EXC_ARITHMETIC	注册异常端口 task_set_ex
② BSD 层（POSIX 信号级）	BSD Signal	UNIX/POSIX 兼容层	SIGSEGV、SIGABRT、SIGBUS、SIGILL	注册信号回调：signal() / s
③ Objective-C 层（运行时级）	NSException	Foundation 框架	[NSException raise:...]	NSSetUncaughtExceptio

三层异常的触发路径，就是从底向上的。也就是如果Mach Exception没有处理就会交由BSD Signal层，BSD Signal没有处理，就会交由NSException处理。

3层异常捕获的原理：

- **Mach Exception捕获机制**
  - 注册自定义异常端口：task_set_exception_port();
  - 当异常发生时，KSCrash的端口接收到消息；
  - KSCrash启动一个专用 “monitor thread” 线程监听这些异常消息；
  - 收到异常后，使用 `thread_get_state()` 获取寄存器状态；
  - 调用栈回溯（通过 `unwind` 或 `backtrace`）；
  - 将异常数据序列化为 JSON crash report；
- **BSD Signal捕获机制**
  - 原理：Mach 异常如果没有被拦截，会转换成 BSD Signal：

例如：

Mach 异常	转换后的 Signal
EXC_BAD_ACCESS	SIGSEGV
EXC_BAD_INSTRUCTION	SIGILL
EXC_ARITHMETIC	SIGFPE
EXC_EMULATION	SIGEMT
EXC_SOFTWARE	SIGABRT

KSCrash 同时通过 `signal()` 注册了信号回调：

```
signal(SIGSEGV, handleSignal);
signal(SIGBUS, handleSignal);
signal(SIGABRT, handleSignal);
```

当信号触发时：

a. 系统执行到 KSCrash 的 `handleSignal()`；

b. **handler 捕获寄存器状态**（通过 `ucontext_t`）；

c. **异步安全地记录堆栈、寄存器、线程信息**；

d. 写入崩溃文件；

e. 然后调用之前的 signal handler 或终止进程。

⚙️ 特点

- 在信号 handler 中**不能调用任何非异步安全函数**；
- 不能使用 malloc、objc_msgSend 等；
- 所以只能记录最小量信息；
- 这部分信息后来由 KSCrash “Crash Sentry” 模块组合上报。

- **NSException异常捕获**

代码块

```
NSSetUncaughtExceptionHandler(&handleUncaughtException);
```

注册语言层异常捕获。

当一个未被 try/catch 捕获的 `[NSException raise:reason:]` 发生时，系统会调用这个回调。

KSCrash 在这里能做的事更多：

a. 获取 `exception.name`，`reason`，`callStackSymbols`；
b. 收集线程状态（因为此时仍安全可用 ObjC）；
c. 记录当前 app 状态（视图层级、主线程调用栈、内存使用等）；
d. 写入崩溃报告文件。

> 由于此时堆栈未损坏，可以调用 Foundation 层 API，因此能生成更丰富的上下文数据。

堆栈获取

这里以KSCrash为例：

> **KSCrash 通过 3 个步骤获取堆栈：**
> 1. **在崩溃发生时抓取 CPU 寄存器快照（PC/FP/LR/SP）**
> 2. **基于 FP（Frame Pointer）向上遍历函数栈帧链表**
> 3. **对每个线程重复该过程，形成完整 backtrace**

1、获取寄存器（崩溃点 CPU 状态）

通过监听崩溃的函数，首先可以获取到寄存器快照：

信息	作用
PC	崩溃的准确代码地址
FP（x29）	指向当前函数栈帧
LR（x30）	返回地址
SP	当前栈顶

2、使用 FP 做链表式栈帧遍历（Backtrace）

ARM64 栈帧结构如下：

```
FP → [上一个 FP]
     [返回地址 LR]
     [局部变量…]
SP → ...
```

因此可以：

```
frame[0] = PC
frame[1] = LR
frame[2] = *(FP+8)
frame[3] = *(previousFP+8)
...
```

KSCrash 用的文件：

KSStackCursor_Backtrace.c

核心伪代码：

```
代码块
cursor->stack[0] = registers.pc;
cursor->stack[1] = registers.lr;
fp = registers.fp;
while(fp != NULL) {
    next_fp = *fp;
    return_addr = *(fp + 8);
    cursor->stack.push(return_addr);
    fp = next_fp;
}
```

非常原始、非常干净。

3、对每个线程都做一次（线程枚举）

KSCrash 会 dump 全部线程

4、写入 JSON（KSCrashReportWriter）

KSCrash 并不会符号化，只保存地址，例如：

```
代码块
"backtrace": [
  { "instruction_addr": "0x1023ac590" },
  { "instruction_addr": "0x1023ac110" },
  { "instruction_addr": "0x19bc1c5d8" }
]
```

符号化在 **客户端上传后，服务端完成**。

总结（KSCrash 堆栈获取的核心能力）

KSCrash 获取堆栈 = 寄存器快照 + 栈帧链表遍历 + Fallback 机制

完整链路：

1. **崩溃 → 捕获寄存器上下文**
2. **FP/LR 做链表回溯（最核心）**
3. **遍历全部线程生成 backtrace**

4. **将地址写入 JSON**
5. **后续再符号化 → 变成函数名和行号**

这是业内最接近系统底层的实现（除了自己直接写 Mach port handler）。

堆栈符号化：

kSCrash JSON的记录是这样的：

```
代码块
1   "instruction_addr": "0x1023ac590"
```

这是一个**纯内存地址**。

符号化就是把地址转换成：

```
代码块
1   具体函数名 + 源文件 + 行号
2   如：-[ChatMessageCell layoutSubviews] (ChatMessageCell.m:145)
```

符号化的底层原理（非常重要）

1️⃣ 根据地址找出它属于哪一个 Mach-O / image（可执行文件/动态库）

iOS APP 有成百上千个 image：

- 主 executable
- Framework（AFNetworking、SDWebImage…）
- 系统库（UIKit、Foundation…）

堆栈 JSON 会记录：

```
代码块
1   image_base: 0x102380000
2   image_size: ...
3   image_name: "MyApp"
4   instruction_addr: 0x1023AC590
```

确定：
instruction_addr 落在哪个 image 的地址范围

2️⃣ 根据地址 - image 基址 = 偏移量

offset = instruction_addr - image_base

假设：

```
代码块

1   image_base = 0x102380000
2   instruction = 0x1023AC590
3   offset = 0x2C590
```

这个 offset 是关键，它在 **dSYM 的内容里有线索**。

3️⃣ 在 dSYM 的 DWARF 调试信息中查找这个 offset 属于哪个 symbol

dSYM(真实存调试符号) 中包含：

- 所有函数的起始地址
- 所有编译单元（源文件）
- 每行代码对应的地址范围

符号化工具需要从 DWARF 里做匹配：

```
代码块

1   offset ∈ symbol 的可执行地址范围 → 找到函数名
2   offset ∈ line-table 的地址范围  → 找到行号
```

4️⃣ 符号化工具链

- `atos`
- `llvm-dwarfdump`
- `symbolicatecrash`
- PLCrashReporter 的 PLCrashSymbolicator
- KSCrash 一套 symbolicator（偏地址解析为主，不含行号）
- 自研 APM 后端一般用 llvm + dwarfdump

特殊崩溃

野指针 / Zombie 崩溃

野指针不好定位，因为1它真正的崩溃对象已经被释放甚至内存被覆盖了，甚至线程都不在当前线程。

1、常见高频野指针危险案例：

以下案例，基本可以通过SwiftLint规则和AI CR规范，在开发阶段避免。

- 1、deinit中访问lazy属性或者刷新UI的行为，因为deinit已经是self或者视图标记销毁的阶段。
- 2、NotificationCenter观察者未移除
  - 解决方案：在deinit中移除，或使用block。
- 3、KVO未移除
- 4、timer强持有
- 5、代理、闭包强引用
- 6、GCD中的self引用，最容易出问题。

代码块

```
// ❌ 危险：GCD持有self
func fetchData() {
    DispatchQueue.global().async {
        let data = self.processData()  // ⚠️ 强引用self
        DispatchQueue.main.async {
            self.updateUI(data)        // ⚠️ 强引用self
        }
    }
}

// ✅ 正确
func fetchData() {
    DispatchQueue.global().async { [weak self] in
        guard let self = self else { return }
        let data = self.processData()
        DispatchQueue.main.async { [weak self] in
            self?.updateUI(data)
        }
    }
}
```

!!这个案例很适合举例子，比如切到后台做一些耗时的操作，然后回到主线程刷新UI。而且这种DispatchQueue是单向引用，不会造成内存泄露，一般不会加weak。所以就容易出现self被销毁了，但是仍然调用刷新视图的操作，导致野指针崩溃问题。

2、线上监控到的一个越狱设备hook引发的野指针崩溃案例：

```
# Crashlytics - Stack trace
# Application: com.hiclub.gravity
# Platform: apple
# Version: 13.1.1 (12657)
# Issue: 2667f4bf9b6d3a0f2892c21ebf84ee4c
# Session: fa69e73fe9c3492ca1a14f393c6cb198_DNE_0_v2
# Date: Fri Oct 31 2025 15:19:15 GMT+0800 (中国标准时间)

com.apple.main-thread
0   libsystem_kernel.dylib          0xaac mach_msg_trap + 8
1   libsystem_kernel.dylib          0x107c mach_msg + 72
2   CoreFoundation                  0x6d78 __CFRunLoopServiceMachPort + 368
3   CoreFoundation                  0xb080 __CFRunLoopRun + 1160
4   CoreFoundation                  0x1e240 CFRunLoopRunSpecific + 572
5   GraphicsServices                0x1988 GSEventRunModal + 160
6   UIKitCore                       0x4e541c -[UIApplication _run] + 1080
7   UIKitCore                       0x27eb88 UIApplicationMain + 336
8   GRAVITY                         0xe76208 main + 23 (AppDelegate.swift:23)
9   ???                             0x10581c3d0 (缺少)

ellekit_exc_port
0   libsystem_kernel.dylib          0xaac mach_msg_trap + 8
1   libsystem_kernel.dylib          0x107c mach_msg + 72
2   libellekit.dylib                0x1c8cc $s7ellekit16ExceptionHandlerC8portLoopyyACSgFZ +
144
3   libellekit.dylib                0x1c7f8
$s7ellekit16ExceptionHandlerC13startPortLoopyyFyyYbcfU_ + 60
4   libellekit.dylib                0x1cc68 $sIegh_IeyBh_TR + 28
5   libdispatch.dylib               0x63094 _dispatch_call_block_and_release + 24
6   libdispatch.dylib               0x64094 _dispatch_client_callout + 16
7   libdispatch.dylib               0x3bab8 _dispatch_continuation_pop$VARIANT$armv81 + 432
8   libdispatch.dylib               0x3b1f4 _dispatch_async_redirect_invoke + 580
9   libdispatch.dylib               0x48630 _dispatch_root_queue_drain + 328
10 libdispatch.dylib                0x48dec _dispatch_worker_thread2 + 160
11 libsystem_pthread.dylib          0x1dd4 _pthread_wqthread + 224
12 libsystem_pthread.dylib          0x193c start_wqthread + 8

tunnel_msg_loop
0   libsystem_kernel.dylib          0x1484 __psynch_cvwait + 8
1   libsystem_pthread.dylib         0x2860 _pthread_cond_wait$VARIANT$armv81 + 1224
2   GRAVITY                         0x28f040c ThreadHandler::msg_loop(void*) + 360
(ff_ffmsg_queue.h:360)
3   GRAVITY                         0x295f8d8 SDL_RunThread + 34 (ijksdl_thread_ios.m:34)
4   libsystem_pthread.dylib         0x3348 _pthread_start + 116
5   libsystem_pthread.dylib         0x1948 thread_start + 8

Thread
```

野指针崩溃.txt

如何判断是否是野指针：

- Mach Exception崩溃在EXC_BAD_ACCESS
- 然后崩溃地址是一个不存在）0x0

> 某个对象 / 函数指针已被释放，调用时跳到了无意义的地址（UAF）。

代码块

```
EXC_BAD_ACCESS KERN_INVALID_ADDRESS 0x0000000100000000
```

然后发现它崩溃的队列是在 ies.safe.guard.queue。这是一个广告SDK（字节穿山甲SDK）

代码块

```
 Crashed: ies.safe.guard.queue
0  ???                              0x100000000 (缺少)
```

而且发现这台用户设备是越狱设备。因为发现它有ellekit插件：

> ellekit 是 iOS 越狱环境里的 Hook 框架，替代 Cydia Substrate。

代码块

```
ellekit_exc_port
libellekit.dylib
```

所以分析大概原因就是： 越狱插件（ellekit）注入，hook 你某个对象/函数失败，导致函数指针被破坏 → 后台线程调用时跳到了 0x100000000 → 崩溃。

处理方案：

- 升级广告SDK版本
- 加强越狱检测
- 屏蔽有灰产记录的用户
- 整体野指针检测与质量

治理难度极高：

除了上述常见现象的崩溃，其他野指针的崩溃治理是非常难的。

野指针崩溃 → 基本无法定位精确代码 → 真正治理靠“减少发生概率 + 数据下降判断”。

这是所有大厂产线的做法。

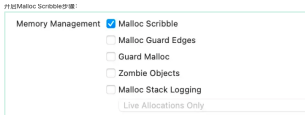
如何治理或防止野指针崩溃：

线下复现野指针

- **Malloc Scribble**

  原理：官方文档说对象在申请内存 alloc 时在内存上填0xAA，释放内存 dealloc 在内存上填0x55，当访问这两种内存就会crash

- 实现流程：
  - 开启 malloc scribble



利用这个增加野指针崩溃的概率：

代码块

```
void safe_free(void* p){
    size_tmemSiziee=malloc_size(p);
    memset(p,0x55, memSiziee);
    orig_free(p);
    return;
}
```

在safe_free方法中对已经释放变量的内存，填充0x55，使已经释放变量不能访问，从而使某些野指针的crash从不必现安变成必现。

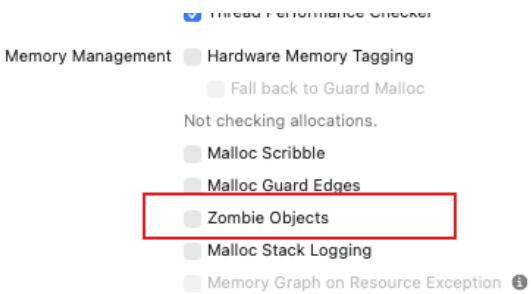
- 为了防止填充0x55的内存被新的数据内容填充，使野指针crash变成不必现，在这里采用的策略是，safe_free不释放这片内存，而是自己保留着，即safe_free方法中不会真的调用free。
- 同时为了防止系统内存过快消耗（因为要保留内存），需要在保留的内存大于一定值时释放一部分，防止被系统杀死，同时，在收到系统内存警告时，也需要释放一部分内存

发生crash时，得到的崩溃信息有限，不利于问题排查，所以这里采用代理类（即继承自NSProxy的子类），重写消息转发的三个方法，以及NSObject的实例方法，来获取异常信息。但是这的话，还有一个问题，注意NSProxy只能做OC对象的代理.

- **Zombie Objects**

原理：一个对象已经解除了它的引用，已经被释放掉，但是此时仍然是可以接受消息，这个对象就叫做Zombie Objects（僵尸对象）。这种方案的重点就是将释放的对象，全都转为僵尸对象。

- 实现流程：
  - 开启Zombie



- 对象释放时，如果当前开启了zombie 设置，会进入__dealloc_zombie函数，内部会生成[_NSZombie_类名]的类，将当前类isa指向[_NSZombie_类名]类，伪代码：

代码块

```
// 获取到即将deallocted对象所属类（Class）
Class cls = object_getClass(self);

// 获取类名
const char *clsName = class_getName(cls)

// 生成僵尸对象类名
const char *zombieClsName = "_NSZombie_" + clsName;

// 查看是否存在相同的僵尸对象类名，不存在则创建
Class zombieCls = objc_lookUpClass(zombieClsName);
if (!zombieCls) {
    // 获取僵尸对象类 _NSZombie_
    Class baseZombieCls = objc_lookUpClass("_NSZombie_");

    // 创建 zombieClsName 类
    zombieCls = objc_duplicateClass(baseZombieCls, zombieClsName, 0);
}

// 在对象内存未被释放的情况下销毁对象的成员变量及关联引用。
objc_destructInstance(self);

// 修改对象的 isa 指针，令其指向特殊的僵尸类
objc_setClass(self, zombieCls);
```

- 最简单最重要，不要犯以下错误，然后可以通过SwiftLint和CR等来规范代码
  - 异步回调未取消
  - Timer 未 invalid
  - Delegate 未置 nil
  - Notification 未移除
  - Swift Task 未取消
  - C++ SDK 回调晚于 destroy

线上野指针

刚才说到线上野指针很难定位，其次类似线下那种Zombie操作，也是不现实的，会引入大量内存。所以只能做一些轻量级的类似Matrix SafeObj（线上轻量）的方案，

原理：就是跟进对象的生命周期，然后推断“哪个对象提前释放”。注意：它不是实时的。

- Hook 1：拦截 objc_release / objc_retain（最核心）
  - Hook 后 SafeObj ，记下“这个对象被释放了”：
    - 对象地址
    - 对象 class
    - 释放时堆栈
    - 释放线程
    - 对象大小
    - 对象所在 malloc 堆区
    - 释放时间戳（精确到 ms）

    然后调用原始 `objc_release` 继续正常释放。

    ⚠ SafeObj 并不会阻止释放，它只是在释放前“做一份犯罪记录”。就像：每个对象被 dealloc 时 SafeObj 都偷偷给你打个“死亡档案”。

    这些档案放在一个 **环形缓冲区（Ring Buffer）** 里（不会占太多内存）。
- Hook 2：Hook dealloc（弱 Hook）
  - 这里不是真正替换 dealloc（否则就变成 Zombie 了，会 OOM）。而是：

    代码块
    ```
    class_replaceMethod(cls, dealloc, my_dealloc)
    ```

    但 **my_dealloc 不会阻止 free，也不会保留对象**。它只做一件事：✔ 记录“最后一次存活的堆栈”

    因为 release 可能不会导致 dealloc（对象可能被强引用住）。

    所以 Hook dealloc 用来补全信息：
    - 谁创建了这个对象？
    - 在哪里释放干净？
    - 最后一次活着时的上下文是什么？
- Hook 3：在崩溃发生时，检查 fault_address 是否“像是某个被释放的对象”
  - 通过崩溃捕获的地址，SafeObj会去环形缓冲区查找：最近 3 秒内有没有释放过一个对象，它的地址非常接近这个地址？

    查找条件包括：
    - 地址是否落在 heap 划分区域？
    - 是否 align（对象头部必须按 8 字节对齐）？

- 地址是否落在已知的 malloc chunk?
- 死亡时间是否很近（< 5s）？
- 大小是否符合对象布局?
- 内存内容是否呈现“涂鸦模式（0x55）”？
- 周围内存是否被其他对象覆盖?

如果匹配足够多（比如超过 70% 匹配度），SafeObj 就会认为：你访问的这个地址，很可能是刚刚释放的那个对象。于是它将：

- 访问堆栈（崩溃的现场）
- 对象的释放堆栈（凶手）
- 对象的创建堆栈（来源）

全部合并到 crash report。

总结：

SafeObj = 野指针“事后还原系统”，不是“实时防护系统”

它依赖：

- Hook retain/release/dealloc
- 创建“死亡档案”
- 崩溃时做地址匹配

⭐ 它不能 100% 定位 UAF，也不能恢复 Zombie，但能提供“很接近真因的线索”

这就是为什么 SafeObj 能在线上使用，但 Zombie 不行。

野指针捕获理论篇

iOS 16键盘野指针崩溃实际案例

OOM / FOOM 崩溃

一定要看：内存优化 —— 开发中解决过最难的问题是什么?

常见的崩溃与防护：

iOS常见的崩溃与防护